

COMPUTER SCIENCE

Complete Scientific Coursebook

Terminale / Year 13 - 80 pages - Explanations in English

Core coverage

- Computer systems and data representation
- Algorithms, Python programming and data structures
- Databases, networking, web technologies and cloud
- Cybersecurity, artificial intelligence and practical projects

Theory + worked examples + programming + practical tasks + revision

Purpose and Learning Outcomes

This coursebook introduces the main concepts expected in a senior secondary Computer Science program. It combines theoretical understanding with practical programming and system design. Examples use Python because its syntax is readable, but the underlying ideas apply across languages.

By the end of the course, learners should be able to explain how computers represent data and execute instructions, design algorithms, write and test programs, choose appropriate data structures, use relational databases, describe networking and web technologies, and evaluate security and AI systems.

KEY POINTS

- Explain core computer architecture and data representation.
- Design algorithms using sequence, selection and iteration.
- Write structured Python programs using functions and data structures.
- Use SQL to query relational databases.
- Explain networks, web applications and cloud services.
- Apply cybersecurity principles and discuss responsible AI.

Lessons

01. What Is Computer Science?	6
02. Computational Thinking	7
03. Algorithms and Pseudocode	8
04. Problem Specifications and Requirements	9
05. Testing, Errors and Debugging	10
06. Ethics in Computing	11
07. Computing Careers and Teamwork	12
08. Binary Numbers	13
09. Hexadecimal	14
10. Signed Integers and Two's Complement	15
11. Floating-Point Numbers	16
12. Text Encoding: ASCII and Unicode	17
13. Images and Color	18
14. Audio and Sampling	19
15. Compression and File Formats	20
16. CPU Architecture	21
17. Fetch-Decode-Execute Cycle	22
18. Memory: RAM, ROM and Cache	23
19. Secondary Storage	24
20. Input and Output Devices	25
21. Operating Systems	26
22. Processes and Multitasking	27
23. Virtualization and Cloud Computing	28
24. Embedded Systems and IoT	29
25. System Performance	30
26. Programming Languages	31
27. Variables and Data Types	32
28. Input, Output and Expressions	33
29. Selection: if, elif, else	34
30. Iteration: for and while Loops	35

Table of Contents

Complete Computer Science Coursebook - explanations in English

CONTENTS

Lessons

31. Functions and Procedures	36
32. Strings	37
33. Lists and Arrays	38
34. Dictionaries and Records	39
35. Files and Persistent Data	40
36. Exceptions and Defensive Programming	41
37. Object-Oriented Programming	42
38. Recursion	43
39. Programming Style and Documentation	44
40. Version Control with Git	45
41. Algorithm Efficiency and Big O	46
42. Linear Search	47
43. Binary Search	48
44. Sorting: Bubble, Selection and Insertion	49
45. Stacks and Queues	50
46. Trees and Graphs	51
47. Hashing and Hash Tables	52
48. Abstract Data Types and Choosing Structures	53
49. Database Concepts	54
50. Relational Design and Normalization	55
51. SQL: SELECT Queries	56
52. SQL: INSERT, UPDATE and DELETE	57
53. Transactions, Integrity and Backup	58
54. Network Fundamentals	59
55. IP Addresses, DNS and Routing	60
56. TCP, UDP and Ports	61
57. The Web: HTTP, HTML, CSS and JavaScript	62
58. Client-Server, APIs and Cloud Services	63
59. Cybersecurity Fundamentals	64
60. Artificial Intelligence and Machine Learning	65

Study Method

Each lesson starts with a guiding question, followed by a concise explanation, key ideas, a concept table and either a worked example or a practical application. The final section asks you to recall, apply and evaluate what you learned.

For programming lessons, type the examples yourself rather than only reading them. Change inputs, predict outputs, test boundary cases and deliberately introduce small errors so that debugging becomes a normal part of learning.

KEY POINTS

- Read the guiding question before the explanation.
- Write short notes in your own words.
- Trace algorithms by hand.
- Run and modify code examples.
- Complete the practical labs after the relevant modules.
- Use the revision tests without notes, then check the answer guide.

What Is Computer Science?

Complete Computer Science Coursebook - explanations in English

FOUNDATIONS

GUIDING QUESTION

What problems does computer science study?

CORE EXPLANATION

Computer science studies computation: how information can be represented, processed and communicated through algorithms and machines. It includes theory, programming, hardware, networks, data, security, artificial intelligence and the social impact of computing.

KEY IDEAS

- Computing is more than using devices; it includes designing methods for solving problems.
- An algorithm is a finite sequence of precise steps for a task.
- Programs implement algorithms in a language a computer can execute.
- Abstraction hides unnecessary detail so complex systems can be understood in layers.

CONCEPT TABLE

Concept	Meaning	Example
Algorithm	step-by-step method	sorting names
Program	coded implementation	Python script
Abstraction	simplified model	file icon hides disk details

APPLICATION / EXAMPLE

A navigation app combines algorithms, data structures, networking, databases and user interfaces.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can complex problems be made manageable?

CORE EXPLANATION

Computational thinking is a disciplined approach to problem solving. It commonly uses decomposition, pattern recognition, abstraction and algorithm design. The aim is to produce a solution that can be explained, tested and, when appropriate, automated.

KEY IDEAS

- Decomposition breaks a large task into smaller subproblems.
- Pattern recognition identifies similarities that allow reuse of solutions.
- Abstraction focuses on relevant features while ignoring unnecessary detail.
- Algorithmic thinking expresses a repeatable sequence of steps.

CONCEPT TABLE

Concept	Meaning	Example
Decomposition	split problem	separate login, search, payment
Pattern	repeated structure	same validation rule
Abstraction	focus on essentials	model a student by ID and name

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why write an algorithm before coding?

CORE EXPLANATION

Pseudocode expresses logic in a language-independent form. It allows programmers to reason about inputs, outputs, decisions and repetition before worrying about syntax. Good algorithms are unambiguous, finite and effective.

KEY IDEAS

- Inputs are values supplied to the algorithm.
- Outputs are produced results.
- Selection chooses between alternatives.
- Iteration repeats a block of steps.
- Trace tables help verify behavior step by step.

CONCEPT TABLE

Concept	Meaning	Example
Sequence	steps in order	read -> calculate -> print
Selection	conditional branch	IF score >= 50
Iteration	repeat actions	FOR each item

WORKED CODE EXAMPLE

```
INPUT mark
IF mark >= 50 THEN
  OUTPUT "Pass"
ELSE
  OUTPUT "Review"
END IF
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

What should be known before building a solution?

CORE EXPLANATION

A clear specification describes what a system must do and the constraints under which it must operate. Requirements may be functional, such as generating a report, or non-functional, such as security, response time and accessibility.

KEY IDEAS

- Functional requirements describe services or behavior.
- Non-functional requirements describe quality constraints.
- Inputs, outputs and validation rules should be explicit.
- Ambiguous requirements create costly errors later.

CONCEPT TABLE

Concept	Meaning	Example
Functional	what system does	calculate invoice
Non-functional	quality constraint	load in under 2 seconds
Constraint	limit on design	must work offline

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How do programmers find and correct faults?

CORE EXPLANATION

Testing checks whether a program behaves as expected. Syntax errors prevent code from being parsed, runtime errors occur during execution, and logic errors produce incorrect results. Debugging is the process of locating and correcting faults.

KEY IDEAS

- Normal test data checks typical valid cases.
- Boundary data checks values at important limits.
- Invalid data tests validation.
- A reproducible test case should include input, expected output and actual output.

CONCEPT TABLE

Concept	Meaning	Example
Syntax error	breaks language grammar	missing colon
Runtime error	fails during execution	division by zero
Logic error	runs but wrong result	incorrect formula

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why do technical choices have ethical consequences?

CORE EXPLANATION

Computing systems influence privacy, access, employment, communication and decision making. Ethical analysis considers stakeholders, benefits, harms, fairness, consent, accountability and environmental impact.

KEY IDEAS

- Privacy concerns how personal data is collected and used.
- Bias can enter through data, design choices or deployment context.
- Accessibility aims to make systems usable by people with diverse needs.
- Professional responsibility includes security and truthful communication of limitations.

CONCEPT TABLE

Concept	Meaning	Example
Privacy	control over personal information	location data
Fairness	avoid unjust disadvantage	balanced decision rules
Accountability	responsibility for outcomes	audit trail

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How is software built by teams?

CORE EXPLANATION

Modern computing projects usually involve multiple roles. Developers, testers, designers, data specialists, security staff and project managers collaborate using shared requirements, version control and communication practices.

KEY IDEAS

- Roles overlap, but responsibilities should be clear.
- Version control records changes and supports collaboration.
- Code review helps detect defects and improve maintainability.
- Documentation allows others to understand and continue the work.

CONCEPT TABLE

Concept	Meaning	Example
Developer	implements software	writes modules
Tester	verifies behavior	runs test plan
Analyst	studies requirements	models processes

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why do digital computers use binary?

CORE EXPLANATION

Digital circuits reliably distinguish two states, commonly represented as 0 and 1. Binary is base 2: each position has a value that is a power of two. A group of eight bits is one byte.

KEY IDEAS

- Binary place values are 1, 2, 4, 8, 16 and so on.
- To convert binary to decimal, add the place values containing 1.
- Unsigned n-bit values range from 0 to $2^n - 1$.
- Bits are also used for flags, colors, instructions and encoded text.

CONCEPT TABLE

Concept	Meaning	Example
Bit	single binary digit	0 or 1
Nibble	4 bits	1010
Byte	8 bits	11001010

APPLICATION / EXAMPLE

$$101101_2 = 32 + 8 + 4 + 1 = 45_{10}$$

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why is hexadecimal useful?

CORE EXPLANATION

Hexadecimal is base 16 and uses digits 0-9 and A-F. One hexadecimal digit corresponds exactly to four binary bits, so long binary patterns can be written more compactly.

KEY IDEAS

- A=10, B=11, C=12, D=13, E=14, F=15.
- Convert binary to hex by grouping bits in sets of four.
- Hex is widely used for memory addresses, color values and machine-level diagnostics.

CONCEPT TABLE

Concept	Meaning	Example
Binary	base 2	1111
Decimal	base 10	15
Hexadecimal	base 16	F

APPLICATION / EXAMPLE

Binary 1101 0110 = D6 in hexadecimal.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How are negative integers stored?

CORE EXPLANATION

Two's complement allows positive and negative integers to use the same binary addition circuitry. In n bits, values range from $-2^{(n-1)}$ to $2^{(n-1)}-1$. The most significant bit has negative weight in the value interpretation.

KEY IDEAS

- To form a negative value, invert the positive bits and add 1.
- Overflow occurs when a result lies outside the representable range.
- Fixed-width arithmetic discards carry beyond the chosen bit width.

CONCEPT TABLE

Concept	Meaning	Example
Unsigned 8-bit	0 to 255	11111111=255
Signed 8-bit	-128 to 127	11111111=-1
Overflow	outside range	127 + 1 in signed 8-bit

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can computers represent fractions and very large values?

CORE EXPLANATION

Floating-point representation stores a number using a sign, a significand and an exponent, conceptually similar to scientific notation. Because the number of bits is finite, many real values are approximations rather than exact values.

KEY IDEAS

- Floating point provides a large range but finite precision.
- Rounding error can accumulate in repeated calculations.
- Comparing decimal fractions for exact equality can be unsafe in many languages.
- Critical financial calculations often use fixed-point or decimal types.

CONCEPT TABLE

Concept	Meaning	Example
Sign	positive/negative	-
Significand	precision digits	1.2345
Exponent	scale	$\times 10^6$

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How is text stored as numbers?

CORE EXPLANATION

Text encodings assign numeric codes to characters. ASCII covers a limited set of English characters. Unicode defines code points for characters from many writing systems and symbols; UTF-8 is a widely used Unicode encoding.

KEY IDEAS

- Encoding converts characters to byte sequences.
- Decoding interprets bytes using an expected encoding.
- Mismatched encodings can produce corrupted text.
- UTF-8 uses one to four bytes per Unicode code point.

CONCEPT TABLE

Concept	Meaning	Example
ASCII	legacy character set	A = 65
Unicode	global character repertoire	é, □, Ω
UTF-8	Unicode byte encoding	web text

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How are digital images represented?

CORE EXPLANATION

Raster images are grids of pixels. Each pixel stores color values, often red, green and blue components. Image quality depends on resolution and color depth, while file size can be reduced by compression.

KEY IDEAS

- Higher resolution means more pixels.
- Color depth determines how many color values can be represented.
- Metadata may store dimensions, date, camera or other information.
- Vector graphics store shapes mathematically instead of pixel grids.

CONCEPT TABLE

Concept	Meaning	Example
Resolution	pixel dimensions	1920 x 1080
Color depth	bits per pixel	24-bit RGB
Vector	shape instructions	logo paths

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How is sound converted into digital data?

CORE EXPLANATION

Digital audio samples the amplitude of a sound wave at regular time intervals. Sample rate controls how often measurements are taken, and bit depth controls the precision of each measurement.

KEY IDEAS

- Higher sample rate can capture higher-frequency detail.
- Higher bit depth provides more amplitude levels.
- Uncompressed audio size grows with sample rate, bit depth, channels and duration.
- Compression can be lossless or lossy.

CONCEPT TABLE

Concept	Meaning	Example
Sample rate	samples per second	44.1 kHz
Bit depth	bits per sample	16-bit
Channels	independent audio streams	stereo=2

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why compress data?

CORE EXPLANATION

Compression reduces storage and transmission requirements by exploiting redundancy or discarding less important information. Lossless methods allow exact reconstruction; lossy methods trade some fidelity for smaller files.

KEY IDEAS

- Lossless compression is used when exact recovery is required.
- Lossy compression is common for images, audio and video.
- File extensions suggest formats but do not guarantee content.
- Compression ratio compares original size with compressed size.

CONCEPT TABLE

Concept	Meaning	Example
Lossless	exact reconstruction	PNG, ZIP
Lossy	some information removed	JPEG, MP3
Codec	encode/decode method	video codec

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

What happens inside the processor?

CORE EXPLANATION

The CPU executes machine instructions using the fetch-decode-execute cycle. Key components include the control unit, arithmetic logic unit, registers and buses. Performance depends on many factors, not clock speed alone.

KEY IDEAS

- Program counter identifies the next instruction.
- Instruction register holds the current instruction.
- ALU performs arithmetic and logical operations.
- Registers are very fast storage inside the CPU.

CONCEPT TABLE

Concept	Meaning	Example
Control unit	coordinates execution	decode instruction
ALU	arithmetic and logic	add, compare
Register	small fast storage	program counter

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How does a CPU run a program?

CORE EXPLANATION

The processor repeatedly fetches an instruction from memory, decodes its operation and operands, executes it, and updates state. This cycle is the foundation of stored-program computing.

KEY IDEAS

- Fetch transfers the next instruction into the CPU.
- Decode determines what operation is requested.
- Execute performs the operation and may read or write memory.
- Branches modify which instruction is fetched next.

CONCEPT TABLE

Concept	Meaning	Example
Fetch	obtain instruction	memory -> IR
Decode	interpret opcode	identify ADD
Execute	perform action	update register

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why are several types of memory needed?

CORE EXPLANATION

Computer systems use a memory hierarchy. Cache is small and very fast, RAM is larger working memory, and secondary storage is much larger but slower. ROM or firmware memory stores persistent startup instructions.

KEY IDEAS

- RAM is volatile: contents are lost when power is removed.
- Cache reduces average time to access frequently used data.
- Firmware provides low-level startup and hardware control.
- Memory capacity and latency affect performance differently.

CONCEPT TABLE

Concept	Meaning	Example
Cache	very fast, small	CPU L1 cache
RAM	working memory	open programs
Firmware	persistent startup code	UEFI

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How do SSDs and hard drives differ?

CORE EXPLANATION

Secondary storage retains data without power. Hard disk drives use magnetic spinning platters; solid-state drives use flash memory. SSDs generally provide lower latency and better resistance to physical shock.

KEY IDEAS

- HDDs are often cheaper per gigabyte.
- SSDs have no moving mechanical parts.
- Storage endurance, speed, capacity and cost are design trade-offs.
- Backups are still necessary regardless of storage type.

CONCEPT TABLE

Concept	Meaning	Example
HDD	magnetic disk	large archive
SSD	flash storage	laptop system drive
Backup	separate recoverable copy	external or cloud copy

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How does a computer interact with the world?

CORE EXPLANATION

Input devices capture data or commands; output devices present results or control external systems. Sensors and actuators are important in embedded and automated systems.

KEY IDEAS

- Input can be manual, automatic or sensor-based.
- Output can be visual, audio, printed or physical action.
- Device drivers translate operating-system requests to hardware-specific operations.
- Accessibility devices broaden ways of interacting with systems.

CONCEPT TABLE

Concept	Meaning	Example
Input	data into system	keyboard
Output	data from system	display
Sensor	measures environment	temperature sensor

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

What services does an operating system provide?

CORE EXPLANATION

An operating system manages hardware resources and provides common services for applications. Core responsibilities include process scheduling, memory management, file systems, device management, security and user interfaces.

KEY IDEAS

- Processes compete for CPU time and other resources.
- Virtual memory extends the usable address space.
- File systems organize persistent data.
- Permissions control which users or processes can access resources.

CONCEPT TABLE

Concept	Meaning	Example
Process	running program instance	browser process
File system	organizes stored files	NTFS/ext4
Scheduler	allocates CPU time	time slice

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can many programs appear to run at once?

CORE EXPLANATION

A multitasking operating system rapidly switches the CPU among processes or threads. On multi-core processors, some tasks genuinely run simultaneously. Scheduling balances responsiveness, fairness and throughput.

KEY IDEAS

- A process has its own execution state and resources.
- A thread is a unit of execution within a process.
- Context switching saves and restores execution state.
- Race conditions can occur when concurrent tasks share mutable data.

CONCEPT TABLE

Concept	Meaning	Example
Process	independent execution context	text editor
Thread	execution path in process	worker thread
Context switch	change running task	scheduler action

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can one physical machine host many logical systems?

CORE EXPLANATION

Virtualization abstracts physical computing resources so multiple virtual machines or containers can share infrastructure. Cloud computing provides configurable resources over networks and often uses virtualization at scale.

KEY IDEAS

- Virtual machines emulate complete computer environments.
- Containers isolate applications while sharing the host kernel.
- Cloud services can provide infrastructure, platforms or software.
- Scalability and cost must be balanced with security and vendor dependence.

CONCEPT TABLE

Concept	Meaning	Example
VM	virtual computer	test server
Container	isolated app environment	Docker container
Cloud	remote computing resources	hosted database

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

What makes embedded computing different?

CORE EXPLANATION

An embedded system is designed for a dedicated function inside a larger product. IoT devices add network connectivity so sensors and actuators can exchange data. These systems often have strict limits on power, memory, cost and real-time behavior.

KEY IDEAS

- Embedded software may run continuously for years.
- Real-time systems must respond within timing constraints.
- Security updates are essential for network-connected devices.
- Sensors collect data; actuators change the physical environment.

CONCEPT TABLE

Concept	Meaning	Example
Embedded system	dedicated computing	washing machine controller
IoT	networked device	smart meter
Actuator	produces physical action	motor

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How should computer performance be evaluated?

CORE EXPLANATION

Performance depends on workload. CPU speed, core count, cache, memory capacity, storage latency, network bandwidth and software efficiency can all matter. Benchmarks should reflect the tasks users actually perform.

KEY IDEAS

- Latency measures delay for one operation.
- Throughput measures work completed per unit time.
- A bottleneck is the resource limiting overall performance.
- Optimization should target measured bottlenecks rather than assumptions.

CONCEPT TABLE

Concept	Meaning	Example
Latency	time per task	10 ms response
Throughput	tasks per time	100 requests/s
Bottleneck	limiting component	slow disk

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why are there many programming languages?

CORE EXPLANATION

Programming languages provide different abstractions and trade-offs. High-level languages emphasize readability and portability; low-level languages expose hardware details. Languages can be compiled, interpreted, or use hybrid execution models.

KEY IDEAS

- Syntax defines valid program structure.
- Semantics describes meaning.
- Compilers translate source code before execution.
- Interpreters execute or translate code during runtime.

CONCEPT TABLE

Concept	Meaning	Example
High-level	human-readable abstraction	Python
Low-level	close to machine	assembly
Compiler	translates program	C compiler

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How does a program store values?

CORE EXPLANATION

Variables associate names with values. Data types define what values are allowed and what operations are meaningful. Common types include integers, floating-point numbers, Booleans, strings and collections.

KEY IDEAS

- Choose types that match the data and required operations.
- Type conversion changes a value from one representation to another.
- Constants represent values that should not change.
- Clear variable names improve readability.

CONCEPT TABLE

Concept	Meaning	Example
Integer	whole number	42
Boolean	true/false	is_logged_in
String	text sequence	"Hello"

WORKED CODE EXAMPLE

```
name = "Aina"
age = 17
is_student = True
print(name, age, is_student)
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How do programs transform input into output?

CORE EXPLANATION

Expressions combine values and operators to produce results. Input functions capture external values, while output functions present results. Operator precedence determines the order in which parts of an expression are evaluated.

KEY IDEAS

- Arithmetic operators include +, -, *, / and %.
- Comparison operators produce Boolean results.
- Parentheses make evaluation order explicit.
- Input should be validated before use.

CONCEPT TABLE

Concept	Meaning	Example
Arithmetic	numeric calculation	price * qty
Comparison	tests relation	age >= 18
Boolean	combines conditions	a and b

WORKED CODE EXAMPLE

```
price = float(input("Price: "))
qty = int(input("Quantity: "))
total = price * qty
print("Total:", total)
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How does a program make decisions?

CORE EXPLANATION

Selection executes different blocks depending on conditions. Conditions should be mutually understandable, correctly ordered and tested at important boundaries.

KEY IDEAS

- if checks the first condition.
- elif tests additional conditions when earlier ones are false.
- else handles the remaining case.
- Nested selection is possible but excessive nesting can reduce clarity.

CONCEPT TABLE

Concept	Meaning	Example
if	first test	if score >= 80
elif	additional test	elif score >= 50
else	fallback	else: review

WORKED CODE EXAMPLE

```
if score >= 80:  
    grade = "A"  
elif score >= 50:  
    grade = "Pass"  
else:  
    grade = "Review"
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How do programs repeat work?

CORE EXPLANATION

Loops repeat instructions. A for loop is useful when iterating over a sequence or known range; a while loop repeats while a condition remains true. Every loop needs a clear termination condition.

KEY IDEAS

- Counter-controlled loops repeat a known number of times.
- Condition-controlled loops repeat until a state changes.
- Infinite loops occur when termination is never reached.
- Loop invariants help reason about correctness.

CONCEPT TABLE

Concept	Meaning	Example
for loop	iterate over sequence	for x in items
while loop	repeat by condition	while balance > 0
break	exit loop early	stop on sentinel

WORKED CODE EXAMPLE

```
total = 0
for n in [4, 7, 9]:
    total += n
print(total)
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why divide programs into functions?

CORE EXPLANATION

Functions encapsulate reusable behavior. They can accept parameters and return values. Breaking a program into small functions improves testing, readability, reuse and maintenance.

KEY IDEAS

- Parameters are names used inside the function.
- Arguments are values supplied when calling it.
- Return values send results back to the caller.
- A function should ideally have one clear responsibility.

CONCEPT TABLE

Concept	Meaning	Example
Parameter	function input name	radius
Argument	supplied value	5
Return value	result from function	area

WORKED CODE EXAMPLE

```
def area_circle(radius):  
    return 3.14159 * radius * radius  
  
print(area_circle(5))
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can programs work with text?

CORE EXPLANATION

Strings are sequences of characters. Common operations include indexing, slicing, concatenation, searching, splitting and formatting. Because text may contain many languages, modern programs should handle Unicode correctly.

KEY IDEAS

- String indexing accesses individual positions.
- Slicing extracts ranges.
- Methods transform or search text.
- Formatting inserts values into readable messages.

CONCEPT TABLE

Concept	Meaning	Example
Index	single position	word[0]
Slice	subsequence	word[1:4]
Format	combine text and values	f"Total {x}"

WORKED CODE EXAMPLE

```
name = "Computer Science"
print(name.lower())
print(name[0:8])
print(len(name))
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can many related values be stored together?

CORE EXPLANATION

Lists or arrays store ordered collections of values. They allow indexed access and iteration. Some languages use fixed-size homogeneous arrays, while others provide dynamic lists.

KEY IDEAS

- Indexes usually begin at 0.
- Traversal processes each element.
- Appending adds a new item to a dynamic list.
- Choosing a structure affects performance and clarity.

CONCEPT TABLE

Concept	Meaning	Example
Index	element position	items[0]
Traverse	visit each item	for item in items
Append	add to end	items.append(x)

WORKED CODE EXAMPLE

```
scores = [62, 75, 81, 49]
passed = [s for s in scores if s >= 50]
print(passed)
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can structured data be represented?

CORE EXPLANATION

A dictionary maps keys to values. Records or objects group related fields into one entity. These structures make programs more expressive than using unrelated variables.

KEY IDEAS

- Dictionary keys should uniquely identify entries.
- Records group fields such as name, ID and score.
- Nested structures can represent complex data.
- Data validation remains necessary even with structured types.

CONCEPT TABLE

Concept	Meaning	Example
Dictionary	key-value mapping	student["name"]
Record	group of fields	Student(id,name)
Nested data	structure inside structure	list of dictionaries

WORKED CODE EXAMPLE

```
student = {"name": "Miora", "score": 84}
print(student["name"])
student["passed"] = student["score"] >= 50
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can a program keep data after it closes?

CORE EXPLANATION

Files store data on secondary storage. Programs open files, read or write content, and close resources safely. Common formats include plain text, CSV and JSON.

KEY IDEAS

- Text files are simple and human-readable.
- CSV represents tabular data.
- JSON represents structured key-value and list data.
- Use context managers or equivalent constructs to ensure files are closed.

CONCEPT TABLE

Concept	Meaning	Example
Text	plain characters	notes.txt
CSV	rows and columns	students.csv
JSON	structured data	config.json

WORKED CODE EXAMPLE

```
with open("notes.txt", "w", encoding="utf-8") as f:  
    f.write("Computer Science\n")
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How should software respond to unexpected conditions?

CORE EXPLANATION

Defensive programming anticipates invalid input, missing resources and exceptional states. Exception handling allows a program to recover, report an error clearly or terminate safely.

KEY IDEAS

- Validate assumptions at system boundaries.
- Catch only exceptions you can handle meaningfully.
- Error messages should help users correct the problem.
- Logging records diagnostic information for later analysis.

CONCEPT TABLE

Concept	Meaning	Example
Validation	check input	age between 0 and 120
Exception	runtime abnormal condition	file not found
Logging	record event details	error log

WORKED CODE EXAMPLE

```
try:
    n = int(input("Number: "))
    print(100 / n)
except (ValueError, ZeroDivisionError):
    print("Enter a non-zero integer.")
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can software model entities with data and behavior?

CORE EXPLANATION

Object-oriented programming organizes code around objects that combine state and behavior. Classes define reusable structures, while objects are instances of classes.

KEY IDEAS

- Encapsulation groups data and methods.
- Inheritance can reuse or extend behavior, but composition is often simpler.
- Polymorphism allows different objects to respond through common interfaces.
- Classes should represent meaningful abstractions.

CONCEPT TABLE

Concept	Meaning	Example
Class	blueprint	Student
Object	class instance	student1
Method	behavior on object	student.average()

WORKED CODE EXAMPLE

```
class Student:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"Hello, {self.name}"
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can a function solve a problem using smaller versions of itself?

CORE EXPLANATION

Recursion occurs when a function calls itself. A recursive solution must include a base case that stops further calls and a recursive case that reduces the problem.

KEY IDEAS

- Base cases prevent infinite recursion.
- Each recursive call should move toward the base case.
- Recursion naturally fits trees and divide-and-conquer algorithms.
- Iteration may use less memory for simple repetitive tasks.

CONCEPT TABLE

Concept	Meaning	Example
Base case	stopping rule	$n \leq 1$
Recursive case	smaller subproblem	<code>factorial(n-1)</code>
Call stack	stores active calls	nested recursion

WORKED CODE EXAMPLE

```
def factorial(n):  
    if n <= 1:  
        return 1  
    return n * factorial(n - 1)
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

What makes code maintainable?

CORE EXPLANATION

Readable programs use clear naming, consistent formatting, small functions and useful documentation. Comments should explain purpose or reasoning rather than repeat obvious syntax.

KEY IDEAS

- Names should reveal intent.
- Avoid duplicated logic by extracting reusable functions.
- Documentation should describe interfaces and assumptions.
- Automated formatters and linters improve consistency.

CONCEPT TABLE

Concept	Meaning	Example
Naming	descriptive identifiers	total_price
Comment	explain reasoning	why threshold exists
Refactor	improve structure	extract function

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How are code changes tracked safely?

CORE EXPLANATION

Version control records the history of source files and supports collaboration. Git uses commits, branches and merges so developers can work independently and integrate changes systematically.

KEY IDEAS

- A commit is a recorded snapshot with a message.
- A branch is an independent line of development.
- A merge combines histories.
- Conflicts require human decisions when edits overlap.

CONCEPT TABLE

Concept	Meaning	Example
Commit	recorded change	fix validation
Branch	parallel development	feature/login
Merge	combine changes	integrate feature

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why can two correct algorithms perform very differently?

CORE EXPLANATION

Complexity analysis describes how time or memory grows as input size increases. Big O notation focuses on the dominant growth pattern, allowing algorithms to be compared independently of a specific computer.

KEY IDEAS

- $O(1)$ is constant growth.
- $O(\log n)$ grows slowly and often appears in divide-and-conquer search.
- $O(n)$ grows proportionally with input size.
- $O(n^2)$ can become expensive for large inputs.

CONCEPT TABLE

Concept	Meaning	Example
$O(1)$	constant	array index
$O(n)$	linear	scan list
$O(n^2)$	quadratic	nested pair comparison

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

When is a simple sequential search appropriate?

CORE EXPLANATION

Linear search checks items one by one until the target is found or the collection ends. It works on unsorted data and is easy to implement, but its worst-case time grows linearly with the number of items.

KEY IDEAS

- Best case: target is first item.
- Worst case: target is last or absent.
- Time complexity is $O(n)$.
- Useful for small or unsorted collections.

CONCEPT TABLE

Concept	Meaning	Example
Input	collection + target	[4,7,9], 7
Output	position/found flag	index 1
Complexity	linear	$O(n)$

WORKED CODE EXAMPLE

```
def linear_search(items, target):
    for i, value in enumerate(items):
        if value == target:
            return i
    return -1
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why does sorted data allow faster searching?

CORE EXPLANATION

Binary search repeatedly compares the target with the middle element of a sorted collection and discards half of the remaining search space. This gives logarithmic time complexity.

KEY IDEAS

- The data must be sorted according to the same ordering used by the search.
- Each step halves the remaining interval.
- Worst-case time complexity is $O(\log n)$.
- Off-by-one errors are common in manual implementations.

CONCEPT TABLE

Concept	Meaning	Example
Requirement	sorted data	1,4,7,9,12
Strategy	halve interval	check middle
Complexity	logarithmic	$O(\log n)$

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How do basic sorting algorithms differ?

CORE EXPLANATION

Elementary sorting algorithms provide useful examples of algorithm design. Bubble sort swaps adjacent out-of-order values; selection sort repeatedly chooses the next smallest; insertion sort inserts each value into an already sorted prefix.

KEY IDEAS

- These algorithms are generally $O(n^2)$ in common worst cases.
- Insertion sort performs well on small or nearly sorted data.
- Sorting improves later searching and reporting.
- Stable sorting preserves the order of equal-key records.

CONCEPT TABLE

Concept	Meaning	Example
Bubble	swap neighbors	teaching algorithm
Selection	choose minimum	simple in-place sort
Insertion	insert into prefix	good nearly sorted

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How do order rules affect data processing?

CORE EXPLANATION

A stack follows last-in, first-out order; a queue follows first-in, first-out order. These abstract data types appear in parsing, undo history, task scheduling and breadth-first search.

KEY IDEAS

- Stack operations commonly include push and pop.
- Queue operations include enqueue and dequeue.
- Underflow occurs when removing from an empty structure.
- Implementation can use arrays, lists or linked structures.

CONCEPT TABLE

Concept	Meaning	Example
Stack	LIFO	browser back history
Queue	FIFO	print jobs
Deque	both ends	task scheduling

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can relationships be represented beyond simple lists?

CORE EXPLANATION

Trees represent hierarchical relationships, while graphs represent general networks of vertices and edges. These structures model file systems, organizations, routes, social networks and dependencies.

KEY IDEAS

- A tree has a root and parent-child relationships.
- A graph may be directed or undirected.
- Edges may carry weights such as distance or cost.
- Traversal algorithms systematically visit nodes.

CONCEPT TABLE

Concept	Meaning	Example
Tree	hierarchy	folder structure
Graph	network	road map
Edge weight	cost/value	distance

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can data be retrieved quickly by key?

CORE EXPLANATION

A hash function maps a key to an index. Hash tables use this mapping for fast average-case insertion, lookup and deletion. Collisions occur when different keys map to the same location and must be resolved.

KEY IDEAS

- Good hash functions distribute keys evenly.
- Collisions are normal and need a strategy.
- Average lookup can be close to $O(1)$.
- Load factor affects performance.

CONCEPT TABLE

Concept	Meaning	Example
Hash function	key -> index	hash("Aina")
Collision	same index	two keys collide
Load factor	table occupancy	items/capacity

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How should a data structure be selected?

CORE EXPLANATION

A data structure should match the operations that matter most. Choices depend on access pattern, ordering, insertion/deletion frequency, memory limits and the need for key-based retrieval.

KEY IDEAS

- Lists are simple for ordered collections.
- Sets emphasize unique membership.
- Dictionaries support key-based lookup.
- Queues and stacks encode order constraints.

CONCEPT TABLE

Concept	Meaning	Example
List	ordered sequence	scores
Set	unique membership	tags
Map	key-value lookup	user by ID

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why use a database instead of many separate files?

CORE EXPLANATION

A database stores related data in a structured system that supports reliable querying, updates, security and concurrency. A database management system provides rules and tools for managing that data.

KEY IDEAS

- Tables organize records and fields in relational databases.
- Primary keys uniquely identify rows.
- Foreign keys connect related tables.
- Constraints help preserve data quality.

CONCEPT TABLE

Concept	Meaning	Example
Table	rows and columns	Students
Primary key	unique row ID	student_id
Foreign key	reference to another table	class_id

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can duplicate and inconsistent data be reduced?

CORE EXPLANATION

Relational design separates entities into related tables. Normalization reduces unnecessary duplication and update anomalies by ensuring attributes depend on appropriate keys.

KEY IDEAS

- First normal form uses atomic values and consistent rows.
- Relationships can be one-to-one, one-to-many or many-to-many.
- Many-to-many relationships usually require a junction table.
- Good design balances integrity, simplicity and performance.

CONCEPT TABLE

Concept	Meaning	Example
Entity	thing being stored	Student
Relationship	association	Student enrolls Course
Junction table	resolves many-to-many	Enrollment

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How is information retrieved from a relational database?

CORE EXPLANATION

SQL is a language for working with relational data. SELECT chooses columns, FROM identifies tables, WHERE filters rows, ORDER BY sorts results, and aggregate functions summarize groups.

KEY IDEAS

- Use explicit column names when possible.
- WHERE filters before grouping.
- GROUP BY forms groups for aggregate calculations.
- Parameterized queries prevent SQL injection in applications.

CONCEPT TABLE

Concept	Meaning	Example
SELECT	choose columns	SELECT name
WHERE	filter rows	WHERE score >= 50
ORDER BY	sort result	ORDER BY name

WORKED CODE EXAMPLE

```
SELECT name, score
FROM students
WHERE score >= 50
ORDER BY score DESC;
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How is stored data changed safely?

CORE EXPLANATION

SQL data-modification statements insert, update and delete records. Conditions are crucial: an UPDATE or DELETE without an appropriate WHERE clause can affect every row.

KEY IDEAS

- INSERT adds new records.
- UPDATE changes existing records.
- DELETE removes records.
- Transactions allow a set of changes to succeed or fail together.

CONCEPT TABLE

Concept	Meaning	Example
INSERT	add row	new student
UPDATE	change row	update email
DELETE	remove row	delete inactive record

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How do databases protect correctness?

CORE EXPLANATION

Transactions group related operations and aim to preserve database consistency. Integrity constraints prevent invalid relationships, while backups and recovery plans protect against hardware failure, mistakes and attacks.

KEY IDEAS

- Atomicity means a transaction completes entirely or not at all.
- Consistency preserves defined rules.
- Isolation reduces interference among concurrent transactions.
- Durability means committed changes survive failures.

CONCEPT TABLE

Concept	Meaning	Example
Atomicity	all-or-nothing	bank transfer
Integrity	valid relationships	foreign key
Backup	recoverable copy	daily snapshot

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

Why connect computers into networks?

CORE EXPLANATION

Networks allow devices to share data, services and resources. Local area networks cover limited spaces; wide area networks connect distant locations. Communication depends on agreed protocols.

KEY IDEAS

- Bandwidth is the maximum data transfer capacity.
- Latency is delay between sending and receiving.
- Routers forward packets between networks.
- Switches connect devices within local networks.

CONCEPT TABLE

Concept	Meaning	Example
LAN	local network	school lab
WAN	large geographic network	company branches
Router	forwards packets	home gateway

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How does internet traffic find its destination?

CORE EXPLANATION

IP addresses identify network interfaces. DNS translates domain names into IP addresses. Routers use routing information to forward packets across interconnected networks toward their destination.

KEY IDEAS

- IPv4 uses 32-bit addresses; IPv6 uses 128-bit addresses.
- Private addresses are commonly used inside local networks.
- DNS is hierarchical and distributed.
- Packets may take different routes across the internet.

CONCEPT TABLE

Concept	Meaning	Example
IP address	network identifier	192.0.2.10
DNS	name resolution	example.com -> IP
Route	path through networks	router hops

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How do applications communicate over IP networks?

CORE EXPLANATION

Transport protocols add application-to-application communication on top of IP. TCP provides ordered, reliable byte streams; UDP provides lightweight datagrams without built-in delivery guarantees. Ports identify application endpoints.

KEY IDEAS

- TCP uses acknowledgements and retransmission.
- UDP has lower overhead and can suit real-time traffic.
- Ports distinguish services on one device.
- Protocol choice depends on reliability, latency and application design.

CONCEPT TABLE

Concept	Meaning	Example
TCP	reliable stream	web transfer
UDP	datagrams	real-time media
Port	application endpoint	443 for HTTPS

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

What happens when a web page loads?

CORE EXPLANATION

A browser requests resources using HTTP or HTTPS. HTML provides document structure, CSS controls presentation, and JavaScript adds behavior. Web applications often communicate with servers through APIs.

KEY IDEAS

- HTTPS protects HTTP traffic using TLS.
- HTML is semantic structure, not a programming language.
- CSS separates styling from content.
- JavaScript can respond to user events and update pages dynamically.

CONCEPT TABLE

Concept	Meaning	Example
HTML	page structure	heading, form
CSS	presentation	layout, fonts
JavaScript	behavior	validate form

WORKED CODE EXAMPLE

```
<h1>Computer Science</h1>
<p id="msg">Welcome</p>
<script>
  document.getElementById("msg").textContent = "Ready";
</script>
```

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How do modern applications exchange data?

CORE EXPLANATION

In client-server systems, clients request services from servers. APIs define structured ways for software components to communicate. Cloud platforms host applications, databases, storage and other services that can scale on demand.

KEY IDEAS

- Clients may be browsers, mobile apps or other services.
- REST-style APIs often use HTTP methods and JSON.
- Authentication identifies users or systems.
- Rate limits and validation protect shared services.

CONCEPT TABLE

Concept	Meaning	Example
Client	requests service	mobile app
Server	provides service	web API
API	software interface	JSON endpoint

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

How can systems reduce security risk?

CORE EXPLANATION

Cybersecurity protects confidentiality, integrity and availability of systems and data. Security is a process of risk management using technical controls, policies, user education, monitoring and recovery planning.

KEY IDEAS

- Confidentiality limits unauthorized disclosure.
- Integrity protects against unauthorized modification.
- Availability ensures services remain usable.
- Defense in depth uses multiple independent protections.
- Strong authentication, patching and backups are basic controls.

CONCEPT TABLE

Concept	Meaning	Example
Confidentiality	prevent disclosure	access control
Integrity	prevent tampering	hash verification
Availability	keep service usable	redundancy

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

GUIDING QUESTION

What makes a system "intelligent"?

CORE EXPLANATION

Artificial intelligence is a broad field concerned with systems that perform tasks associated with perception, reasoning, language, prediction or decision making. Machine learning builds models from data rather than encoding every rule manually.

KEY IDEAS

- Supervised learning trains on labeled examples.
- Unsupervised learning looks for structure without target labels.
- Training, validation and test data serve different purposes.
- Model quality depends on data quality, evaluation metrics and deployment context.
- AI systems require attention to bias, privacy, robustness and human oversight.

CONCEPT TABLE

Concept	Meaning	Example
Supervised	learn from labels	spam classifier
Unsupervised	find patterns	clustering
Model	learned mapping	predict score

APPLICATION / EXAMPLE

Apply the concept to a real computer system, program, school network or digital service. Identify the inputs, processing, outputs and any design trade-offs.

CHECK YOUR UNDERSTANDING

1. Define two key terms from this lesson.
2. Explain the guiding question in 3-4 sentences.
3. Give one new example not shown on this page.
4. State one advantage and one limitation or trade-off.
5. Write one exam-style question about this topic.

TASK

Design an algorithm for a school canteen checkout. Inputs: item prices and quantities. Outputs: subtotal, discount if applicable, final total and receipt.

REQUIREMENTS

Create pseudocode first. Include validation for negative quantities. Add at least three test cases: normal, boundary and invalid data.

EXTENSION

Extension: add a student discount and a daily spending limit.

WORKSPACE / DESIGN NOTES

Evidence to submit: design, implementation or diagram, test evidence, and a short evaluation.

TASK

Write a Python program that reads a list of marks and reports count, minimum, maximum, mean and number of passes.

REQUIREMENTS

Use functions. Validate each mark as 0-100. Store valid marks in a list and process them with a loop.

EXTENSION

Extension: output a frequency table for grade bands.

WORKSPACE / DESIGN NOTES

Evidence to submit: design, implementation or diagram, test evidence, and a short evaluation.

Practical Lab 3 - Text File Processing

Complete Computer Science Coursebook - explanations in English

PRACTICAL WORK

TASK

Create a program that reads a UTF-8 text file and reports the number of lines, words and characters.

REQUIREMENTS

Use a context manager to open the file. Handle a missing file gracefully. Test with an empty file and a multilingual file.

EXTENSION

Extension: report the five most frequent words.

WORKSPACE / DESIGN NOTES

Evidence to submit: design, implementation or diagram, test evidence, and a short evaluation.

TASK

Compare linear search with binary search on ordered data.

REQUIREMENTS

Generate increasingly large lists. Measure or count comparisons. Explain why binary search requires sorted data.

EXTENSION

Extension: compare insertion sort with the built-in sort for larger inputs.

WORKSPACE / DESIGN NOTES

Evidence to submit: design, implementation or diagram, test evidence, and a short evaluation.

TASK

Design a small database for courses, students and enrollments.

REQUIREMENTS

Draw the entities and relationships. Define primary keys and foreign keys. Then write CREATE TABLE or pseudocode schema definitions.

EXTENSION

Extension: write SQL to list each student with the courses they take.

WORKSPACE / DESIGN NOTES

Evidence to submit: design, implementation or diagram, test evidence, and a short evaluation.

Practical Lab 6 - Web Page

Complete Computer Science Coursebook - explanations in English

PRACTICAL WORK

TASK

Build a simple educational web page using HTML and CSS, then add a small JavaScript interaction.

REQUIREMENTS

Use semantic headings, paragraphs, a list and a form. Ensure labels are associated with form controls.

EXTENSION

Extension: validate an input before displaying a result.

WORKSPACE / DESIGN NOTES

Evidence to submit: design, implementation or diagram, test evidence, and a short evaluation.

TASK

Plan a small network for a school office with 10 computers, one printer and internet access.

REQUIREMENTS

Draw or describe the topology. Identify switch, router, IP addressing approach, Wi-Fi security and backup connectivity.

EXTENSION

Extension: separate staff and guest traffic logically.

WORKSPACE / DESIGN NOTES

Evidence to submit: design, implementation or diagram, test evidence, and a short evaluation.

TASK

Choose one digital service that uses personal data or AI. Evaluate risks and safeguards.

REQUIREMENTS

Identify data collected, users affected, possible misuse, bias or security risks, and suitable controls.

EXTENSION

Extension: propose an incident response or human-review process.

WORKSPACE / DESIGN NOTES

Evidence to submit: design, implementation or diagram, test evidence, and a short evaluation.

Answer without notes first. Use diagrams, pseudocode or code where appropriate.

1. Convert 10110110_2 to decimal and hexadecimal.

2. Explain one difference between RAM and secondary storage.

3. Describe the fetch-decode-execute cycle.

4. Explain why floating-point values can contain rounding error.

5. Compare lossless and lossy compression.

6. Explain two responsibilities of an operating system.

7. Distinguish latency from throughput.

8. Give one ethical issue caused by large-scale data collection.

Answer without notes first. Use diagrams, pseudocode or code where appropriate.

1. Write pseudocode to find the largest of three numbers.

2. Explain the difference between a syntax error and a logic error.

3. Write a function that returns True when an integer is even.

4. Compare a list and a dictionary.

5. Explain when binary search can be used.

6. State the time complexity of linear search and binary search.

7. Explain the roles of a base case and recursive case.

8. Describe why version control is useful in team development.

Answer without notes first. Use diagrams, pseudocode or code where appropriate.

1. Define primary key and foreign key.

2. Write an SQL query to select names of students with score ≥ 60 .

3. Explain what DNS does.

4. Compare TCP and UDP.

5. Explain the purpose of HTTPS.

6. Define confidentiality, integrity and availability.

7. Distinguish supervised and unsupervised machine learning.

8. Give two reasons why AI evaluation must consider deployment context.

1. Binary 10110110 = 182 decimal = B6 hexadecimal.
2. RAM is volatile working memory; secondary storage is non-volatile and usually larger but slower.
3. Fetch gets the instruction, decode identifies the operation and operands, execute performs it and updates state.
4. Floating point uses finite bits, so many real numbers are approximated.
5. Lossless allows exact reconstruction; lossy discards some information for smaller files.
6. Operating-system roles include process scheduling, memory management, files, devices and security.
7. Latency is delay per operation; throughput is amount of work per unit time.
8. Possible issues include privacy, consent, surveillance, bias and security.
9. Largest-of-three pseudocode may initialize max to the first value then compare the other two.
10. Syntax errors violate language grammar; logic errors run but produce wrong behavior.
11. Python: `def is_even(n): return n % 2 == 0`
12. Lists use ordered positions; dictionaries retrieve values by keys.
13. Binary search requires data sorted according to the search ordering.
14. Linear search $O(n)$; binary search $O(\log n)$.
15. Base case stops recursion; recursive case reduces the problem.
16. Version control tracks history, supports branches, collaboration and rollback.

1. Primary key uniquely identifies a row; foreign key references a key in another table.
2. `SELECT name FROM students WHERE score >= 60;`
3. DNS translates domain names into IP addresses.
4. TCP provides reliable ordered delivery; UDP provides lightweight datagrams without built-in reliability.
5. HTTPS protects HTTP traffic with TLS encryption and server authentication.
6. Confidentiality prevents unauthorized disclosure; integrity prevents unauthorized alteration; availability keeps services usable.
7. Supervised learning uses labeled targets; unsupervised learning searches for structure without target labels.
8. AI behavior can change with population, incentives, data quality, human workflow and environment, so deployment context matters.
9. Practical work should show not only a final result but also planning, testing and evaluation.
10. For code, include meaningful names, validation, comments only where helpful, and evidence of boundary testing.
11. For database and network tasks, diagrams and assumptions should be clear.
12. For security and AI tasks, identify stakeholders, risks and realistic controls.

Algorithm	finite sequence of steps for solving a problem
Abstraction	simplification that hides unnecessary detail
Bit	binary digit, 0 or 1
Byte	group of 8 bits
CPU	processor that executes instructions
RAM	volatile working memory
Operating system	software managing hardware and common services
Variable	named storage or reference for a value
Function	reusable block of code with defined behavior
Iteration	repetition of instructions
Data structure	organized way to store and access data
Database	structured collection of related data
SQL	language for relational database operations
Protocol	agreed rules for communication
IP address	identifier used for network communication
DNS	system mapping names to network addresses
Encryption	transformation protecting readable data from unauthorized access
Hash	fixed-size value derived from input data
Authentication	process of proving identity
Machine learning	methods that build predictive or descriptive models from data

Core Logic

- Sequence: execute steps in order
- Selection: choose branch using a condition
- Iteration: repeat steps
- Function: reusable named behavior

Python Reminders

- Assignment: `x = 5`
- Condition: `if x > 0:`
- Loop: `for item in items:`
- Function: `def f(x): return x * 2`

Data and Algorithms

- Linear search: $O(n)$
- Binary search on sorted data: $O(\log n)$
- Dictionary/hash-table lookup: average close to $O(1)$
- Simple nested comparison: often $O(n^2)$

SQL Reminders

- `SELECT` columns FROM table
- `WHERE` filters rows
- `ORDER BY` sorts output
- `JOIN` combines related tables

Network and Security

- DNS: name -> IP address
- TCP: reliable ordered transport
- HTTPS: HTTP protected by TLS
- CIA: confidentiality, integrity, availability

FINAL CHECKLIST

- Can I explain a concept without memorized wording?
- Can I trace an algorithm and predict output?
- Can I write and test a short program?
- Can I justify a data structure or system design choice?
- Can I identify security, ethical and reliability risks?